darkmpc: design report

A dark pool its operator cannot see into

Tave La Rocque

November 2026

1. The problem

A dark pool is a trading venue that does not display its order book. Large investors use dark pools so that a big order does not move the market before it fills. The arrangement depends on trusting the operator, and that trust has been broken: Barclays paid $70 million and Credit Suisse $84.3 million in 2016 to settle charges about how their dark pools were run and described (`docs/prior-work.md`). The operator sees every order, so the operator can misuse every order.

darkmpc removes the operator's view. Traders split each order into secret shares and send one share to each of three servers, run by three independent organisations. The servers run the matching auction jointly with secure three-party computation (MPC) and send each trader only the shares of that trader's own fills. No single server ever sees an order, a total, or a fill. Two servers that collude see everything, and nothing in darkmpc prevents that; the design assumes an honest majority.

This report describes what was built, how it was checked, what it costs and what it does not claim. Each section points to the document with the full detail.

2. Threat model in brief

At most one server is corrupt, fixed before the protocol starts, and any number of traders may be malicious and may collude with it. In **semi-honest mode** the corrupt server follows the protocol and tries to learn from what it sees. In **malicious mode** it may deviate arbitrarily, and the guarantee is security with abort: a deviation that would change an output is caught before anything is revealed. There is no fairness or guaranteed output: a corrupt server can always stop an auction. Abort is also not guaranteed to be unanimous. A corrupt server can make one honest server stop while the other waits for messages that never come, but no honest server outputs a wrong value, and a trader who checks both copies of each share it receives rejects incomplete output.

What traders learn is what the auction itself hands them: their own fills and, in Stage 2, the clearing price when filled. That reveals something about other orders by design (a one-share probe that fills shows that interest exists; the Stage 2 price is always some order's limit). Those leaks belong to the auction rules, not the protocol. The

servers learn the number of orders per batch, because batches are not padded. Full statement: `docs/design/threat-model.md`.

3. The two auctions

Both are periodic call auctions: orders accumulate for a fixed interval and are matched together. Each has a plain reference implementation (`crates/reference`) that defines correct output; the MPC version must reproduce it exactly on every valid input.

Stage 1: midpoint volume matching with minimum fills. Every trade in a symbol executes at the public midpoint of the lit market. Per symbol, the matched volume is $T = \min(\text{buy total}, \text{sell total})$, and each side is filled in arrival order. An order may set a minimum fill; if its fill would be positive but below that minimum, it is removed and the allocation repeats. With a public cap of k passes, a symbol still unsettled after k passes matches nothing. This is the volume-match mechanism of the 2018 MPC dark pool by Cartlidge, Smart and Talibi Alaoui, extended with minimum fills and hidden symbols.

Stage 2: a uniform-price auction with limits. Orders carry limit prices. Per symbol, one clearing price is chosen by maximum executable volume, then minimum surplus (the first two criteria of Cboe Europe Periodic Auctions), then the lowest price, which replaces Cboe's reference-price steps that would need a secret lookup when symbols are hidden. Orders fill by price priority, then time. There are no minimum fills in Stage 2, because removing an order can move the price.

4. The cryptographic core

Sharing. Values live in the ring $\mathbb{Z}_{2^{64}}$, so machine arithmetic is the protocol arithmetic. A value is split as $x = x_0 + x_1 + x_2$ and server i holds (x_i, x_{i+1}): replicated secret sharing, after Araki, Furukawa, Lindell, Nof and Ohara (CCS 2016). Any one server's two pieces are uniformly random. Bits are shared the same way with XOR, 64 to a machine word.

Multiplication costs one message of one word per server: each server combines its pieces locally, adds a mask from pseudorandom streams it shares with its neighbours (AES-128 in counter mode), and sends the result to one neighbour. Everything else (addition, multiplication by a public constant) is local.

Comparison. To decide $a < b$, the servers compute the sign bit of $a - b$, which is valid because all sizes stay below 2^{40} and batches below 2^{17}. The three pieces of a value are turned into bit-shares locally, added with a carry-save step and a Kogge-Stone carry circuit of logarithmic depth, and the top bit is converted back to the ring. A comparison costs 10 rounds and 15 words of traffic per server, independent of the batch size, and all comparisons of a batch run at once.

Oblivious sort. Grouping orders by a hidden symbol needs a sort the servers cannot watch. darkmpc uses the stable radix sort of Asharov, Hamada, Ikarashi, Kikuchi, Nof, Pinkas and Tsuji (CCS 2022). For each key bit, the servers build a secret permutation that sorts by that bit, shuffle it with a fresh random permutation that no

single server knows, open the shuffled version (a uniformly random permutation, so it reveals nothing) and apply it. Each pair of servers applies one third of the random permutation, so the server outside each pair never sees its piece. Stability keeps arrival order within a symbol, which is what time priority needs.

Segmented scans. After sorting, per-symbol sums ("buy volume ahead of this order", "total sell volume in this symbol") are prefix sums that restart at each symbol boundary. They take a logarithmic number of rounds. Several scans with different boundaries are laid end to end and computed as one, which cut Stage 1 from 316 to 280 rounds at 1,000 orders.

The auctions on top. Stage 1 sorts by (symbol, side), computes totals and each order's volume ahead, and runs the k elimination passes with the clamp formula $\text{fill} = \min(\text{size}, \max(0, T - \text{ahead}))$. Stage 2 sorts by (symbol, limit). Demand and supply at every order's own limit are sums over the symbol corrected by sums over its run of equal limits. Because demand falls and supply rises along a symbol, only two candidate prices need comparing, found with one multiplication and one comparison per order instead of a search. Allocation then needs no second sort: sells are already in priority order, and a buy's volume ahead is again a symbol sum minus a run sum. Finally each result is routed back to its order's original position. Details: `docs/design/stage1-mpc.md`, `docs/design/stage2-mpc.md`.

Networking. The servers run as separate processes connected by TLS 1.3 with mutual authentication against a deployment certificate authority (rustls). The pseudorandom keys each pair shares are agreed over those connections at setup.

5. Security against a cheating server

Malicious mode adds a verification layer to the unchanged protocol.

- **Inputs** are checked on shares. Each order's pieces are compared between the two servers that hold copies, every bit must satisfy $b - b^2 = 0$, sizes must be below 2^{40}, and size, minimum fill and limit must be in range. An invalid order becomes a dummy that matches nothing.
- **Openings** are checked: each server receives its missing piece from both servers that hold it and aborts if the copies differ (Eerikson et al., ITC 2020).
- **Every multiplication and AND gate** is recorded and verified in batches before anything is opened, against random triples produced by cut-and-choose and checked in buckets (Furukawa, Lindell, Nof and Weinstein, Eurocrypt 2017).
- **Every shuffle** is MAC-checked. Each value carries αx for a secret key α, shuffled with it, and a random linear combination is verified before the permutation is opened. Without this, a cheating server could tamper with a shuffle and learn a secret from whether the protocol aborted, the attack described by Asharov et al. The MAC arithmetic runs in $\mathbb{Z}_{2^{128}}$, the lifting idea of SPDZ2k (Cramer et al., CRYPTO 2018), because in $\mathbb{Z}_{2^{64}}$ an error of 2^{63} would pass half the time.

Two steps here extend published analyses to a new setting: the product checks in $\mathbb{Z}_{2^{64}}$ instead of $\mathbb{Z}_2$, and the MAC check inside an honest-majority shuffle. Both are proved in `docs/design/ring-proofs.md`. For the product checks, a cheater's winning cases in the ring are a subset of its winning cases in the Boolean game the original

paper bounded, so that bound carries over. For the shuffle, a harmful error passes with probability at most $66 \cdot 2^{-65} < 2^{-58.9}$, and whether the check aborts never depends on a secret. Each malicious run computes its own statistical security from the batches it verified: at least 43.9 bits in every benchmarked configuration, a figure that is conservative by about 6 bits.

These proofs are the author's and have not been reviewed by an expert. A full simulation proof that combines them with the published pieces has not been written.

6. What a server learns: the leakage audit

The semi-honest guarantee is proved by simulation in `docs/design/leakage-audit.md`: a program that sees only the batch size and configuration can produce a server's entire view, and no efficient test can tell it from the real one if AES is a pseudorandom function. The argument is short because there are only four kinds of interaction. A server's input pieces are uniform. Multiplication messages carry a mask from the one key the server lacks. In a shuffle, the server receives values only in the step that excludes it, and they are masked the same way. And the only values it ever sees in the clear are uniformly random permutations, because the one third of the shuffle it does not know randomises them.

Two tests check that the code matches the protocol the proof is about. A **shape test** requires every server's traffic, message by message, to be identical for two very different batches. An **opening test** requires the distribution of every opened value to be the same for both batches, and every opened permutation to be uniform. Two leaks are planted to show these tests can fail: opening the sorting permutation without shuffling it, and a stray debug opening of the clearing price. Both leave every fill correct, so no correctness test can see them; the audit catches both.

7. How it was checked

Correctness and security claims here rest on checks that were built to fail when something is wrong.

- **A reference and differential tests.** The plain matcher defines correct output. Property tests generate random batches (biased towards ties, crossings and boundary values) and require the MPC output to equal the reference's exactly, in both modes, for both stages.
- **Properties of the reference itself:** conservation, time and price priority, maximal volume against a brute-force search over every price, and agreement of Stage 2 with Stage 1 when all limits are equal.
- **Planted bugs.** 27 deliberate bugs live in the code behind feature flags, ranging from a wrong share in multiplication to a cheating server keeping its own share, a skipped input check, or a leak. `scripts/check_planted.sh` builds the workspace once per bug and fails unless every one is caught. A test suite that has never caught anything has not shown it can.
- **Exhaustive proof checks.** The two theorems are checked by brute force on instances small enough to enumerate: every error pattern and every bucket

permutation for the product checks, and the exact best pass probability in small rings for the shuffle check.

What the checks found. The method earned its cost:

- A property test found a two-order counterexample to the Stage 2 crossing short-cut, a buy of 13 at 101 and a sell of 1 at 95, before any cryptography existed.
- Writing the shuffle-check proof showed the permutation sampler was biased by more than the security target allows at the triple-permutation sizes, so it was replaced by rejection sampling.
- An assumption that long carry chains are rare was wrong, because chain length depends on the secret value, so a chosen-input test now covers every length.
- The checksum optimisation hid a dependent network hop that the round counter did not see.
- Real deployments surfaced TCP resets from unread TLS session tickets, `ENOBUFS` on macOS loopback, and memory exhaustion in malicious mode at 10,000 orders. Each was fixed and has a test.

8. Performance

Measured on three Google Cloud `n2-standard-2` VMs (one physical core each), every run reconstructed and checked against the reference, medians of 3. Full tables and method: `docs/benchmarks.md`.

	LAN (0.2 ms, 9.3 Gbit/s)	WAN (27 to 69 ms, 0.75 Gbit/s)
Stage 1, 1,000 orders, semi-honest	0.055 s	7.3 s
Stage 1, 100,000 orders, semi-honest	4.33 s	23.6 s
Stage 1, 1,000 orders, malicious	0.948 s	17.1 s
Stage 1, 10,000 orders, malicious	9.88 s	44.2 s
Stage 2, 1,000 orders, malicious	1.45 s	25.6 s

Rounds grow only logarithmically with the batch size (268 to 308 from 100 to 100,000 orders for Stage 1), and on the WAN each round costs about 25 ms. So a WAN auction takes at least about 6 s however small, and large batches amortise that: semi-honest throughput rises from 17 to 4,233 orders per second between 100 and 100,000 orders. Malicious mode costs 13 to 28 times semi-honest time on the LAN, almost all of it the product checks' extra data.

Against prior work. The only published three-party actively secure dark pool (Cartlidge, Smart and Talibi Alaoui, ePrint 2018/1045, Table 8) reports 0.900 s online plus 4.136 s preprocessing for 1,000 orders, with 4,509 rounds. darkmpc takes 0.948 s with no preprocessing and 566 rounds, while hiding symbols and sides, validating inputs and running minimum-fill passes their mechanism does not have. It also runs on fewer and newer cores, a faster LAN, and different software, so the comparison is between two systems, not two algorithms. Neither earlier paper measured a WAN.

9. Limits

- **Honest majority.** Two colluding servers see everything.

- **Not audited, proofs not reviewed.** The two ring extensions are proved by the author only. There is no end-to-end simulation proof for malicious mode.
- **Security with selective abort.** There is no fairness, no guaranteed output, and abort is not guaranteed to be unanimous (Section 2).
- **Batch size is public.** Padding to a fixed size is not implemented.
- **Side channels.** Message counts and sizes are fixed, but CPU timing and memory access are not audited.
- **Inherent leaks to traders.** Probing ("pinging") and the Stage 2 price being some order's limit cannot be removed by cryptography. They need market rules.
- **No trader client or settlement.** The benchmarks deal shares on behalf of traders. A deployment needs a client that shares orders and checks both copies of every output share.
- **Mechanism simplifications.** Stage 1 trades at a public midpoint whose integrity is assumed. Stage 2 breaks final ties by lowest price, not Cboe's reference-price rules.

10. Where to read more

Topic	Document
Prior work and its verified numbers	`docs/prior-work.md`
Threat model and leakage	`docs/design/threat-model.md`
Stage 1 and Stage 2 in MPC	`docs/design/stage1-mpc.md`, `docs/design/stage2-mpc.md`
Malicious security design	`docs/design/malicious.md`
Leakage proof and audit	`docs/design/leakage-audit.md`
Proofs of the two ring extensions	`docs/design/ring-proofs.md` (and `docs/ring_proofs.pdf`)
Benchmarks	`docs/benchmarks.md`
Concept walkthroughs in prose	`docs/learning/`, and the milestone PDFs in `docs/`

References

- Araki, Furukawa, Lindell, Nof, Ohara. High-Throughput Semi-Honest Secure Three-Party Computation with an Honest Majority. CCS 2016. ePrint 2016/768.
- Asharov, Hamada, Ikarashi, Kikuchi, Nof, Pinkas, Tsuji. Efficient Secure Three-Party Sorting with Applications to Data Analysis and Heavy Hitters. CCS 2022. ePrint 2022/1595.
- Cartlidge, Smart, Talibi Alaoui. MPC Joins The Dark Side. AsiaCCS 2019. ePrint 2018/1045.
- Cartlidge, Smart, Talibi Alaoui. Multi-Party Computation Mechanism for Anonymous Equity Block Trading: A Secure Implementation of Turquoise Plato Uncross. ePrint 2020/662.
- Cramer, Damgård, Escudero, Scholl, Xing. SPDZ2k: Efficient MPC mod 2^k for Dishonest Majority. CRYPTO 2018. ePrint 2018/482.
- Eerikson, Keller, Orlandi, Pullonen, Puura, Simkin. Use your Brain! Arithmetic 3PC For Any Modulus with Active Security. ITC 2020. ePrint 2019/164.
- Furukawa, Lindell, Nof, Weinstein. High-Throughput Secure Three-Party Computation for Malicious Adversaries and an Honest Majority. Eurocrypt 2017. ePrint 2016/944.

- Cboe Europe. Periodic Auctions (participant guidance).